

DRONE GCS

Technical Code Documentation

Autonomous Ground Control Station for UAV Inventory Operations

Project: PAE-EDISA 2025–2026

University: Universitat Politècnica de Catalunya (UPC)

Platform: Raspberry Pi 5 · ROS2 Jazzy · Electron

Version: 1.0

This document describes the internal architecture and code-level behaviour of every module in the Drone GCS system. It is intended for developers and system integrators.

Contents

1	System Overview	3
1.1	Architecture Diagram	3
1.2	Communication Protocols	3
2	Raspberry Pi Stack	3
2.1	rosbridge.service & rosbridge_boot.sh	4
2.1.1	Boot sequence	4
2.2	gcs_control.py — Service Manager Node	4
2.2.1	Design	4
2.2.2	Process lifecycle	5
2.2.3	Topics	5
2.3	camera_publisher.py — Pi Camera ROS2 Node	5
2.3.1	Capture pipeline	5
2.3.2	QoS profile	6
2.4	mjpeg_server.py — Dashboard Camera HTTP Server	6
2.4.1	Why two camera systems?	6
2.4.2	Colour correction	6
2.5	barcode_detector.py — Vision and Inventory Node	6
2.5.1	Processing pipeline	6
2.5.2	Throttling and deduplication	7
2.5.3	SLAM position tagging	7
2.6	mavlink_bridge.py — MAVLink ↔ ROS2 Bridge	7
2.6.1	Threading model	7
2.6.2	Serial port auto-detection	8
2.6.3	Connection sequence	8
2.6.4	GCS heartbeat (F-08 fix)	8
2.6.5	MAVLink message dispatch table	9
2.6.6	Euler to quaternion conversion	9
2.6.7	Command handler	9
2.7	slam_launch.sh — LiDAR + Point-LIO Launcher	9
2.7.1	Startup sequence	9
2.8	brain_node.py — Autonomous Mission Planner	10
2.8.1	Database schema	10
2.8.2	Control loop	10
3	GCS Frontend (Electron Application)	10
3.1	main.js — Electron Shell	11
3.2	renderer/app.js — Application Logic	11
3.2.1	Global state object	11
3.2.2	Connection management	11
3.2.3	Network auto-discovery	12
3.2.4	Dual camera paths	12
3.2.5	ROS2 subscriptions	12
3.2.6	Command flow	12
3.2.7	Service management	13
4	Complete ROS2 Topic Map	13
5	Deployment Quick Reference	13

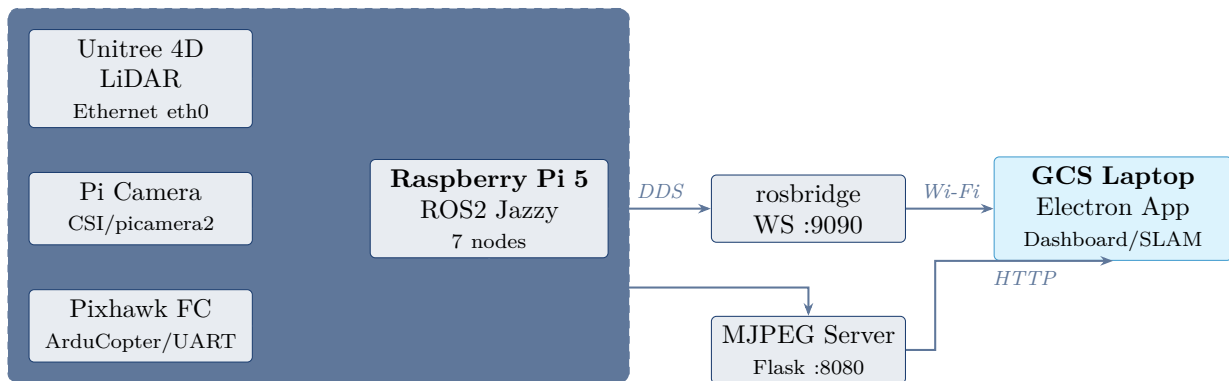
5.1	Raspberry Pi prerequisites	13
5.2	Deploy updated files	14
5.3	GCS app (laptop)	14
5.4	Environment variables	14

1. System Overview

Drone GCS is a full-stack UAV ground control system. The software runs across two physically distinct machines:

- **Raspberry Pi 5 (onboard)** — ROS2 Jazzy node graph. Handles sensor data acquisition, SLAM processing, barcode detection, MAVLink communication with the flight controller, and the autonomous mission planner. Exposes the entire ROS2 topic graph over a WebSocket server (rosbridge) on port 9090.
- **GCS Laptop (ground)** — Electron desktop application. Connects to the Pi over Wi-Fi via rosbridge and visualises all telemetry, camera streams, and SLAM data in real time. No ROS installation is required on the laptop.

1.1. Architecture Diagram



1.2. Communication Protocols

Protocol	Transport	Port	Purpose
rosbridge WebSocket	TCP / Wi-Fi	9090	Full ROS2 topic pub/sub from laptop
MAVLink v2	Serial UART	57600 baud	Flight controller telemetry and commands
MJPEG HTTP	TCP / Wi-Fi	8080	Dashboard live camera stream
ROS2 DDS (RTPS)	UDP / LAN	dynamic	Inter-node communication on the Pi

2. Raspberry Pi Stack

All Python files are deployed to `~/` on the Pi and launched by `gcs_control.py`. The startup chain is:

```
rosbridge.service → rosbridge_boot.sh → gcs_control.py → (dashboard buttons) → individual nodes
```

2.1. rosbridge.service & rosbridge_boot.sh

Role: systemd service that auto-starts on every boot. Launches the rosbridge WebSocket server and then exec's into `gcs_control.py`.

2.1.1. Boot sequence

```

1 # 1. Source the ROS2 environment (Jazzy distro)
2 source /opt/ros/${ROS_DISTRO:-jazzy}/setup.bash
3 source "$HOME/rosbridge_ws/install/setup.bash" 2>/dev/null || true
4 source "$HOME/slam_ws/install/setup.bash"        2>/dev/null || true
5
6 # 2. Launch rosbridge WebSocket server in the background
7 ros2 launch rosbridge_server rosbridge_websocket_launch.xml &
8 BRIDGE_PID=$!
9
10 # 3. Give rosbridge 3 s to bind port 9090 before the control node
11 #    tries to publish its first status message
12 sleep 3
13
14 # 4. Replace this shell process with gcs_control.py (exec -> no subprocess
15 #    overhead)
16 exec python3 "$SCRIPT_DIR/gcs_control.py"

```

Listing 1: rosbridge_boot.sh – startup sequence

The `exec` on the last line is significant: the bash process is replaced by Python, so systemd tracks the correct PID for process supervision.

The `rosbridge.service` unit sets:

- `User=raspi5` — runs as the non-root user
- `Restart=on-failure` with `RestartSec=10` — auto-recovers from crashes
- `After=network-online.target` — waits for Wi-Fi before binding port 9090
- `Environment="ROS_DISTRO=jazzy"` — makes the distro available inside the script

2.2. gcs_control.py — Service Manager Node

Role: ROS2 node that acts as a process manager for all other nodes. Started automatically at boot alongside rosbridge. Receives start/stop commands from the GCS dashboard and manages subprocess lifecycle with clean process-group termination.

2.2.1. Design

`gcs_control.py` is intentionally kept simple and minimal. It does not implement any flight logic — its only job is to translate dashboard button clicks (received as JSON on `/gcs/cmd`) into `subprocess.Popen` calls, and to report running status back at 1 Hz on `/gcs/status`.

```

1 SERVICE_CMDS = {
2     'slam':    ['bash', f'{SCRIPTS}/slam_launch.sh'],
3     'camera': ['python3', f'{SCRIPTS}/camera_publisher.py',
4               '--ros-args', '-p', 'device:=0', ...],

```

```

5  'mjpeg':    ['python3', f'{SCRIPTS}/mjpeg_server.py', '0', '8080', '640', '480', '80'],
6  'mavros':   ['python3', f'{SCRIPTS}/mavlink_bridge.py'],
7  'brain':    ['python3', f'{SCRIPTS}/brain_node.py'],
8  'barcode':  ['python3', f'{SCRIPTS}/barcode_detector.py'],
9  }

```

Listing 2: Service command table in gcs_control.py

2.2.2. Process lifecycle

Each service is started with `preexec_fn=os.setsid` which places the child process in its own process group. When a stop command arrives, `os.killpg()` sends `SIGTERM` to the entire group, ensuring that any grandchild processes (e.g. `slam_launch.sh` and its children) are also terminated cleanly.

```

1  def _stop(self, svc: str):
2      proc = self._procs.get(svc)
3      try:
4          os.killpg(os.getpgid(proc.pid), signal.SIGTERM) # kills the whole group
5          proc.wait(timeout=5)
6      except subprocess.TimeoutExpired:
7          os.killpg(os.getpgid(proc.pid), signal.SIGKILL) # force if still alive

```

Listing 3: Clean process-group termination

2.2.3. Topics

Topic	Direction	Payload
/gcs/cmd	Subscribe	{"action": "start" "stop", "service": "slam" ...}
/gcs/status	Publish @ 1 Hz	{"slam": "running", "camera": "stopped", ...}

2.3. camera_publisher.py — Pi Camera ROS2 Node

Role: Captures frames from the Raspberry Pi Camera Module using `picamera2` and publishes them as `sensor_msgs/CompressedImage` on a ROS2 topic. Consumed by `barcode_detector.py`.

2.3.1. Capture pipeline

```

1  def _publish(self):
2      # Capture a raw frame directly as a numpy array (BGR888 format)
3      frame = self.cam.capture_array()
4      # Encode to JPEG in-memory (no disk I/O)
5      ret, buf = cv2.imencode('.jpg', frame, self._encode_params)
6      if not ret:
7          return
8      msg = CompressedImage()
9      msg.header.stamp = self.get_clock().now().to_msg()
10     msg.header.frame_id = self.fid
11     msg.format = 'jpeg'
12     msg.data = buf.tobytes()
13     self.pub.publish(msg)

```

Listing 4: Camera capture and publish loop

2.3.2. QoS profile

The publisher uses `BEST_EFFORT / VOLATILE / KEEP_LAST(1)`. This is critical: any subscriber to this topic (including `barcode_detector.py`) must use the exact same QoS profile or it will receive no messages — ROS2 DDS enforces QoS compatibility at subscription time.

Why `BEST_EFFORT`? Camera frames are time-sensitive. A reliable (TCP-like) QoS would cause head-of-line blocking if one frame is lost, which would freeze the stream. With `BEST_EFFORT`, dropped frames are skipped silently and the stream continues in real time.

2.4. `mjpeg_server.py` — Dashboard Camera HTTP Server

Role: Serves a Motion-JPEG (MJPEG) stream over HTTP on `:8080/cam1`. Used by the Dashboard tab `<img>` element — no ROS decoding needed on the laptop.

2.4.1. Why two camera systems?

The system uses two separate camera paths:

Path	Technology	Consumer
ROS topic	<code>picamera2</code> → <code>CompressedImage</code>	<code>barcode_detector</code> , <code>icam1</code>
MJPEG HTTP	<code>cv2.VideoCapture (V4L2)</code> → <code>Flask</code>	Dashboard <code>cam1</code>

The MJPEG path is simpler and lower-latency for display. The ROS path provides timestamped, synchronised frames for computer vision processing.

2.4.2. Colour correction

V4L2 on the Raspberry Pi Camera returns pixel data in RGB byte order, but OpenCV's `imencode` expects BGR. A colour swap is applied before encoding:

```

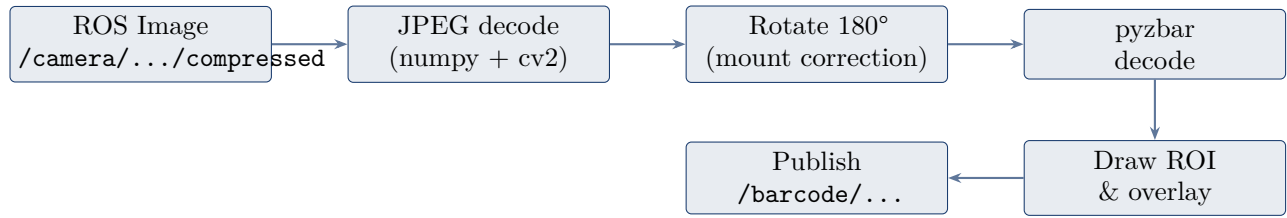
1 ret, frame = cap.read()
2 # V4L2 on RPi Camera returns RGB-ordered data despite cv2 expecting BGR
3 frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
4 _, buf = cv2.imencode('.jpg', frame, encode_params)
```

Listing 5: V4L2 colour order fix in `mjpeg_server.py`

2.5. `barcode_detector.py` — Vision and Inventory Node

Role: Subscribes to the Pi Camera ROS topic, runs barcode and QR code detection on every frame using `pyzbar`, tags each detection with the current SLAM position, and publishes annotated images and detection events.

2.5.1. Processing pipeline



2.5.2. Throttling and deduplication

Two mechanisms prevent flooding downstream consumers:

- **Frame throttle:** Only one frame every 0.25 s (4 fps) is processed. Incoming frames at higher rates are dropped immediately.
- **Detection dedup:** A dict `self._seen{code: timestamp}` suppresses re-publishing the same barcode code for 3 seconds after its first detection. This prevents duplicate database entries when the camera lingers on the same barcode.

2.5.3. SLAM position tagging

The node maintains the latest SLAM pose by subscribing to two topics with automatic fallback:

1. `/Odometry` (Point-LIO primary output) — parsed in `_on_odom()`
2. `/tf` (TF transform tree) — parsed in `_on_tf()`, looking for the `camera_init` → `aft_mapped` transform

Each detection event published to `/barcode/detection` includes:

```

1 {
2   "barcode":    "P003GUA",
3   "code":       "P003GUA",
4   "status":     "OK",
5   "slam_x":     1.600,    # metres in SLAM frame
6   "slam_y":     -2.482,
7   "slam_theta": -116.8   # degrees
8 }

```

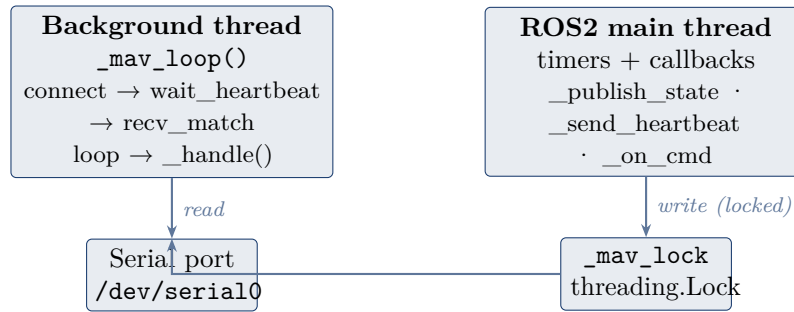
Listing 6: Detection event JSON payload

2.6. mavlink_bridge.py — MAVLink ↔ ROS2 Bridge

Role: Replaces the MAVROS package with a minimal pymavlink bridge. Connects to the Pixhawk over GPIO UART, translates MAVLink messages into ROS2 topics, and forwards GCS commands as MAVLink command packets.

2.6.1. Threading model

The node uses two concurrent execution paths:



`recv_match()` (read) runs exclusively in the background thread and does not need the lock. All writes (`heartbeat_send`, `command_long_send`, `request_data_stream_send`) acquire `_mav_lock` before touching the serial port.

2.6.2. Serial port auto-detection

```

1 candidates = [
2     '/dev/serial0',    # kernel symlink, active when enable_uart=1 is set
3     '/dev/ttyAMA10',  # RPi5 RPi GPIO UART (ttyAMA10 before ttyAMA0)
4     '/dev/ttyAMA0',   # may be Bluetooth on RPi5 -- checked last
5     '/dev/ttyUSB0',   # USB-to-serial adapter
6     '/dev/ttyACM0',   # USB CDC-ACM (Pixhawk USB cable)
7 ]
  
```

Listing 7: Serial port candidate order (F-02 fix applied)

Hardware prerequisite: `enable_uart=1` must be present in `/boot/firmware/config.txt` and a reboot performed before `/dev/serial0` is active on RPi5. Without this, the GPIO UART pins 14/15 are not configured and the bridge will timeout waiting for a heartbeat.

2.6.3. Connection sequence

1. Close previous connection (explicit `close()` prevents fd leak).
2. `mavutil.mavlink_connection(port, baud=57600, source_system=255)`
3. `wait_heartbeat(timeout=15)` — blocks until Pixhawk responds.
4. Extract `target_system` and `target_component` from the heartbeat message fields (py-mavlink may return 0/0 otherwise).
5. Call `_request_streams()` to enable telemetry.
6. Enter blocking `recv_match` loop.

2.6.4. GCS heartbeat (F-08 fix)

A 1 Hz ROS2 timer calls `_send_heartbeat()` which sends `MAV_TYPE_GCS` heartbeats to the Pixhawk. Without this, ArduCopter silences its telemetry streams approximately 5 s after detecting an absent GCS.

2.6.5. MAVLink message dispatch table

MAVLink message	ROS2 topic	Notes
HEARTBEAT	(state cache only)	Armed flag + flight mode
VFR_HUD	/drone/vfr	Speed, altitude, heading
ATTITUDE	/mavros/imu/data	Euler→quaternion convers
SYS_STATUS	/mavros/battery	mV→V, 10mA→A convers
GLOBAL_POSITION_INT	/mavros/global_position/global + velocity_body	lat/lon 1e-7, alt mm, vel cr
GPS_RAW_INT	/drone/gps_status	Fix type, satellite count
SERVO_OUTPUT_RAW	/drone/motors	PWM μ s per channel
RC_CHANNELS	/drone/motors	Fallback if SERVO_OUTPUT

2.6.6. Euler to quaternion conversion

The ATTITUDE message provides roll, pitch, yaw in radians. The bridge converts to unit quaternion using the standard ZYX decomposition:

$$\begin{aligned}
 q &= q_z(\psi) \cdot q_y(\theta) \cdot q_x(\phi) \\
 q_w &= \cos \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} + \sin \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2} \\
 q_x &= \sin \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} - \cos \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2}
 \end{aligned}$$

2.6.7. Command handler

The `_on_cmd` callback deserialises the JSON payload and calls the appropriate MAVLink command. The ARM action requires two clicks within 800 ms (implemented in the frontend) to prevent accidental arming.

2.7. `slam_launch.sh` — LiDAR + Point-LIO Launcher

Role: Replicates the 3-terminal manual SLAM startup sequence as a single automated script. Configures the Ethernet interface for the LiDAR, starts the Unitree driver, waits for LiDAR topics, then starts Point-LIO.

2.7.1. Startup sequence

1. **eth0 configuration:** Assigns static IP 192.168.1.2/24 so the Pi can reach the LiDAR at 192.168.1.62. Uses `sudo -n` to detect whether passwordless `ip` commands are configured in `visudo`.
2. **Ping check:** Non-fatal connectivity test to LiDAR. Script continues regardless (failure is informative only).
3. **unitree_lidar_ros2 driver:** Launches in background. Publishes point cloud on `/unilidar/cloud`.
4. **Topic wait loop:** Polls `ros2 topic list` every second for up to 40 s until a `unilidar` topic appears.
5. **3 s settle:** Gives the LiDAR driver time to initialise its internal ring buffer before Point-LIO starts ingesting data.

6. **Point-LIO:** Launches `mapping_unilidar_12.launch.py` with `rviz:=false` (no display on the headless Pi).

The `trap` at the end ensures both child PIDs receive `SIGTERM` when `gcs_control.py` kills the process group.

2.8. `brain_node.py` — Autonomous Mission Planner

Role: Reads a waypoint mission from a local SQLite database, tracks the drone’s SLAM position, and streams `PoseStamped` setpoints to the flight controller at 10 Hz. Publishes mission progress for the GCS NAV map.

2.8.1. Database schema

```

1 CREATE TABLE missions (
2     id INTEGER PRIMARY KEY,
3     name TEXT NOT NULL,
4     active INTEGER NOT NULL DEFAULT 0, -- 1 = currently loaded
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6 );
7 CREATE TABLE waypoints (
8     id INTEGER PRIMARY KEY,
9     mission_id INTEGER NOT NULL REFERENCES missions(id),
10    seq INTEGER NOT NULL, -- ordering index
11    x REAL NOT NULL, -- SLAM frame metres
12    y REAL NOT NULL,
13    z REAL NOT NULL DEFAULT 5.0,
14    label TEXT DEFAULT '' -- human-readable name (e.g. "origin", "wp-1")
15 );

```

Listing 8: SQLite schema – missions and waypoints

On first launch the node seeds a default 5 m × 5 m square mission if the database is empty. Custom missions can be inserted directly with any SQLite client.

2.8.2. Control loop

Two timers run independently:

- **10 Hz (`_tick`):** Computes 2D Euclidean distance to the current target waypoint. If within `ARRIVE_DIST = 1.5m`, advances `wp_idx` to the next waypoint. Otherwise publishes a `PoseStamped` at the current waypoint position.
- **1 Hz (`_publish_path`):** Publishes the entire mission state as JSON to `/brain/planned_path` for the GCS map to draw.

The two timers are separated intentionally. An earlier version called `_publish_path()` from inside `_tick()`, resulting in 10 JSON messages per second being forwarded over `rosbridge` — causing the map renderer to “go crazy”. Publishing map data at 1 Hz is sufficient for display.

3. GCS Frontend (Electron Application)

3.1. main.js — Electron Shell

Role: Electron main process. Creates the BrowserWindow, loads the renderer HTML, and handles the application lifecycle (quit on close, re-open on macOS activate). Contains no GCS logic.

```

1  const win = new BrowserWindow({
2    width: 1400, height: 820, minWidth: 1280, minHeight: 740,
3    backgroundColor: '#1a1d23',    // dark theme pre-render colour (no white flash)
4    webPreferences: {
5      preload: path.join(__dirname, 'preload.js'),
6      contextIsolation: true,      // renderer cannot access Node.js APIs directly
7      nodeIntegration: false,     // security: renderer is sandboxed
8    },
9  })
10 win.loadFile('renderer/index.html')

```

Listing 9: BrowserWindow creation in main.js

`contextIsolation: true` and `nodeIntegration: false` enforce the security boundary between the renderer (web context) and the main process (Node.js context). Since all communication with the Pi goes via WebSocket (a browser API), no Node IPC bridge is needed and `preload.js` is intentionally empty.

3.2. renderer/app.js — Application Logic

Role: All runtime behaviour of the GCS. Single file, no framework, no build step — vanilla ES2020 running directly in Electron’s Chromium engine. Manages WebSocket connection, all ROS2 subscriptions, canvas rendering, and user interaction.

3.2.1. Global state object

All telemetry data is held in a single mutable `S` object. This acts as the application’s centralised data store, updated by subscription callbacks and consumed by canvas draw functions and DOM updates:

```

1  const S = {
2    speed: 0, vspeed: 0, altitude: 0, heading: 0,
3    roll: 0, pitch: 0, yaw: 0,
4    battery: { pct: 0, voltage: 0 },
5    motors: [0, 0, 0, 0, 0, 0],
6    armed: false, mode: '---',
7    gps: { lat: null, lon: null, fix: 0, sats: 0 },
8    slamPose: { x: 0, y: 0, theta: 0 },
9    plannedPath: [], wpIndex: 0,
10   svcStatus: { slam: 'stopped', camera: 'stopped', mavros: 'stopped', ... },
11 }

```

Listing 10: Shared state object `S` (selected fields)

3.2.2. Connection management

```

1  function connect() {
2    ros = new ROSLIB.Ros({ url: ROS_URL }) // e.g. ws://172.20.10.2:9090
3    ros.on('connection', () => {

```

```

4   setDot('connected')
5   subscribe() // subscribe to all topics on connect
6 }
7   ros.on('close', () => { setDot('error'); sched() }) // retry in 3 s
8 }

```

Listing 11: rosbridge WebSocket connection

The URL is persisted in `localStorage` so the last-used Pi IP is remembered across app restarts.

3.2.3. Network auto-discovery

The connection popover includes a subnet scanner that probes all 254 IPs in the local /24 subnet in parallel batches of 40, with a 400 ms timeout per probe. The local IP is obtained via WebRTC ICE candidates (a technique that works inside Electron without requiring Node.js).

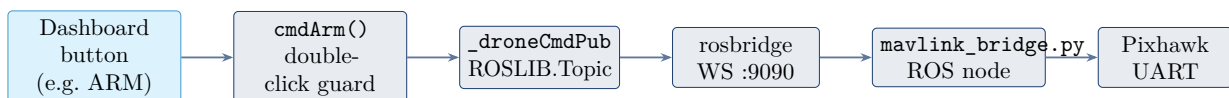
3.2.4. Dual camera paths

Camera	Protocol	Mechanism
Dashboard cam1	MJPEG HTTP	<code><img src="http://PI:8080/cam1" set to http://PI:8080/cam1. On error, retries every 3 s.</code>
Image Processing icam1	ROS2 CompressedImage	Base64 JPEG decoded to <code>Blob</code> → <code>createImageBitmap()</code> → drawn to Canvas in <code>subCamImage()</code> .

3.2.5. ROS2 subscriptions

Topic	Type	Effect on S
/drone/vfr	String (JSON)	S.speed, S.altitude, S.heading, chart history
/drone/state	String (JSON)	S.armed, S.mode
/drone/motors	String (JSON)	S.motors[0..7] (PWM → %)
/drone/gps_status	String (JSON)	S.gps.fix, S.gps.sats
/mavros/battery	BatteryState	S.battery.pct, S.battery.voltage
/mavros/imu/data	Imu	quaternion → Euler: S.roll, S.pitch, S.yaw
/mavros/global_position/global	NavSatFix	S.gps.lat/lon/altAbs, GPS trail
/mavros/local_position/velocity_body	TwistStamped	S.vx, S.vy, S.vz
/brain/planned_path	String (JSON)	S.plannedPath, S.wpIndex, NAV map
/gcs/status	String (JSON)	S.svcStatus → service button colours
/barcode/detection	String (JSON)	S.detections[] → inventory table + DB log
/barcode/roi/image/compressed	CompressedImage	Annotated camera frame in Image Processing tab

3.2.6. Command flow



The ARM command has an additional safety interlock: a first click sets a timestamp; a second click within 800 ms confirms and sends the command. A single click only shows “click again to confirm” in the command log.

3.2.7. Service management

The Services panel sends start/stop JSON to `/gcs/cmd`, which `gcs_control.py` receives and acts on. An additional auto-start rule is applied in the frontend:

```

1 if (svc === 'barcode' && action === 'start') {
2   // Barcode detector needs a camera source -- ensure it is running
3   ['camera', 'camera2', 'mjpeg'].forEach(s => {
4     if (S.svcStatus[s] !== 'running') {
5       _svcPub.publish(new ROSLIB.Message({
6         data: JSON.stringify({ action: 'start', service: s })
7       }))
8     }
9   })
10 }

```

Listing 12: Auto-start cameras when barcode service starts

4. Complete ROS2 Topic Map

Topic	Publisher	Subscriber(s)	Content
/camera/forward/image_raw/compressed	camera_publisher	barcode_detector, app.js	JPEG frames
/barcode/roi/image/compressed	barcode_detector	app.js	Annotated JPEG
/barcode/detection	barcode_detector	app.js	Detection JSON
/drone/vfr	mavlink_bridge	app.js	Speed/alt/heading
/drone/state	mavlink_bridge	app.js	Armed/mode/connected
/drone/motors	mavlink_bridge	app.js	PWM channels
/drone/gps_status	mavlink_bridge	app.js	Fix/satellites
/drone/cmd	app.js	mavlink_bridge	Flight commands
/mavros/battery	mavlink_bridge	app.js	Voltage / %
/mavros/imu/data	mavlink_bridge	app.js	Quat. / ang. vel.
/mavros/global_position/global	mavlink_bridge	brain_node, app.js	GPS NavSatFix
/mavros/local_position/velocity_body	mavlink_bridge	app.js	NED velocities
/mavros/setpoint_position/local	brain_node	Pixhawk	Waypoint target
/brain/planned_path	brain_node	app.js	Mission JSON
/Odometry	Point-LIO	barcode_detector, brain_node	SLAM pose
/tf	Point-LIO	barcode_detector	TF transforms
/gcs/cmd	app.js	gcs_control	Start/stop services
/gcs/status	gcs_control	app.js	Service states

5. Deployment Quick Reference

5.1. Raspberry Pi prerequisites

1. Add `enable_uart=1` to `/boot/firmware/config.txt` and reboot.
2. Install Python dependencies:

```
1 pip3 install --break-system-packages pymavlink pyzbar opencv-python flask
```

3. Install systemd service:

```
1 sudo cp ~/rosbridge.service /etc/systemd/system/  
2 sudo systemctl daemon-reload && sudo systemctl enable --now rosbridge
```

5.2. Deploy updated files

```
1 scp mavlink_bridge.py raspi5@172.20.10.2:~/mavlink_bridge.py
```

Listing 13: SCP individual file to Pi

5.3. GCS app (laptop)

```
1 npm install      # first time only  
2 npm start        # development  
3 npm run build     # create installer (Win / Linux / macOS)
```

5.4. Environment variables

Variable	Default	Effect
MAV_PORT	auto-detect	Override serial port
MAV_BAUD	57600	Override baud rate
ROS_DISTRO	jazzy	ROS2 distro name